Critical Code Studies

electronicbookreview.com/essay/critical-code-studies

January 31, 2012

Mark C. Marino
12-04-2006

Entering the 'cyberdebates' initiated by Nick Montfort, John Cayley, and Rita Raley, new media scholar Mark Marino proposes that we should analyze and explicate code as a text like any other, 'a sign system with its own rhetoric' and cultural embeddedness.

"Hello World" is one of the first programs that computer scientists write in a programming language. The program, usually only a few lines of code, causes the computer to output a greeting, as if it were speaking. The Lisp (List Processing language) version of such a program, for example, looks like this:

(DEFUN HELLO-WORLD ()

(PRINT (LIST 'HELLO 'WORLD)))

DEFUN defines the function HELLO-WORLD (with no set arguments). The computer as a result will speak its greeting to the world in our natural language. What could be a more appropriate language for this activity than Lisp, a family of algebraic list processing languages developed for artificial intelligence (McCarthy 1979)?

But of course, the computer does not understand what it says. Literally speaking, the computer does not even interpret that code. When the function is called, the computer will print (output) the list of the two atoms (as symbolic units are called in Lisp) "Hello" and "World." The single quotation marks tell the computer not to interpret the words "Hello" and "World" (as the double quotation marks do in this sentence). With this distinction, language becomes divided between the operational code and data. The computer here merely shuffles the words as so many strings of data. It does not interpret, only uses those strings. However, those words in quotation marks are significant to us, the humans who read the code. "Hello" and "World" have significance, just as the function name "print" has a significance that goes far beyond its instructions to the computer and gestures toward a material culture of ink and writing surfaces.

Currently, all of computer code lies before us with single quotation marks preceding its lines. While we examine programming architecture and admire modularity and efficiency, the study of computer code does not currently emphasize interpretation, the search for and production of meaning. Even when aesthetics intervene, they come in the form of calls for stylistic clarity for more efficient modification and development. This emphasis on functionality neglects the

meaning that code bears for its human audience. For code, especially mid- to high-level languages, exists not solely for computers, which could operate on machine language (essentially, representations of electronic signals), but for programmers as well. Therefore, the computer may be one recipient of the code but there is also the programmer, other programmers, and at times even users who have access to its text.

To critique code merely for its functionality or aesthetics is to approach code with only a small portion of our analytic tools. In Life on the Screen (1995), Sherry Turkle describes the Julia effect by which interactors ascribe intentionality and sentience to the computer that might display "Hello World" (101). People like to project humanity onto the computer, but is it possible that with regard to coding we do just the opposite and strip the code of its human significance, imagining that it is a sign system within which the extensive analyses of semiotic systems and signification, connotation, and denotation do not apply? Is "Hello World," a rite of passage into computer languages, the beginning of a literacy constrained by restricted interpretation? What would happen if we began to interpret the meaning of the code?

Making the Code the Text

In his posting on the electronic book review, "The Code is Not the Text (unless it is the text)," John Cayley argues against a common notion in new media scholarship, that computer code is the text that we, as humanities scholars, study. According to Cayley, the figure of "the code-as-text," or computer code as the object of interpretation, is "more in the way of decoration or rhetorical flourish, the baroque euphuism of new media." In addition to his own "programmable poetry," Cayley studies the codework of new media artists, such as Talon Memmott and Mez (Mary-Anne Breeze), who has created a poetic language (mesangelle) out of the structures and symbols of computer code. He defines codework as "literature which uses, addresses, and incorporates code: as underlying language-animating or language-generating programming, as a special type of language itself, or as an intrinsic part of the new surface language or 'interface text' in networked and programmable media." However, for Cayley, this last type of codework alone does not suffice.

Cayley's chief complaint is that the analyzed "code" in many of the celebrated codeworks exists merely on the surface of the work, output. By this analysis, critics who discuss representations of coding symbols do so without interpreting code as executable instructions or underlying processes. Because the computer code produced by Mez is not executable, she is dealing with only one aspect of code. As a result, "the code has ceased to function as code." Thus, surface depictions of coding elements are but partial representations, presenting a fraction of code's signifying force.

I'm not sure, though, about the rationale behind Cayley's insistence that code should feature artistic play as well as executable operation. However, executability of the code is not a prerequisite. In response, Rita Raley's "Interferences: [Net.Writing] and the Practice of

Codework" refutes these unnecessary limitations. She compares Cayley's privileging of the code (over the output) to Adorno's "remarks about music as merely a consequence of the score." As she explains, "codework does not suggest, nor does it need to, that code - the algorithmic score, the instructions that govern and produce the system - itself should be privileged."Thus Mez's works are valid codeworks because they play with the structures of code on the display level of the text.

And yet for Raley, the use of coding elements on surface text serves to disrupt our reading (and processing) practices, by means of defamiliarization. It is because this hybrid language of code and natural language cannot be interpreted that it can make familiar processing practices, including reading, strange, producing "a disordering which cannot be predicted" (Shklovsky 1965 qtd. in Raley 2002). Such practice also follows what Raley characterizes as a radical political praxis in codework. In another essay, she has defined "the impetus of contemporary code art" as "to reveal codes, to make the mechanism of production visible to the viewer moving it from background to foreground" (Raley "Code Surface // Code Depth" 2-4). Regardless of whether or not these texts compile at the level of the computer, codework, in Raley's analysis, disrupts the way we as readers compile and interpret these coding symbols in relation to their broader use and operation in the technologies that would otherwise process us without interruption.

Clearly, Cayley, who stresses his identity as a poet, is at once making a critical intervention and also promoting his particular poetics. In his essay, he offers an example of a poem he wrote in a high-level language, "Hypertalk". The piece can be interpreted by humans and processed, or compiled, by computers. It is "working code," which is the only kind of code Cayley considers real. Despite their disagreement over the genre of codework, both Raley and Cayley suggest that code is "a special type of language" that is worthy of the attention of literary scholars. Moreover, because of their emphasis on analyzing code and processes, these codework artists and critics point to a path that leads to the interpretation of code even in programs that are not written as part of literary experiments.

Florian Cramer in Words Made Flesh: Code, Culture, Imagination, offers another formulation of this debate, opening the definition of software to include even non-functioning code.

Software, it follows, is a cultural practice made up of (a) algorithms, (b) possibly, but not necessarily in conjunction with imaginary or actual machines, (c) human interaction in a broad sense of any cultural appropriation and use, and (d) speculative imagination. Software history can thus be told as intellectual history, as opposed to media theories which consider cultural imagination a secondary product of material technology. (124)

Cramer's broader formulation allows him to involve various objects that long predate the saved-state, digital computer. As a result, his choice of objects of studies is far less restricted than Cayley's. He writes,

If the location or even existence of some software isn't quite clear, if a piece software isn't code running on machine because it appears as pseudo code in a book or is, like most Perl poetry for example, not even a working algorithm, then a technical definition of software is too limited. In the end, "software" and "computation" can't be strictly differentiated from each other. The cultural history of software is the cultural history of computation. (124)

While such moves help trace the lines of flight between CCS interpretations and other interpretations of objects within the history of technology, they do, on the other hand, allow Cramer to broaden his objects of study. Such a general, albeit inclusive, definition, however, directs the discussion away from the direct analysis of specific source code.

I would like to propose that we no longer speak of the code as a text in metaphorical terms, but that we begin to analyze and explicate code as a text, as a sign system with its own rhetoric, as verbal communication that possesses significance in excess of its functional utility. While computer scientists can theorize on the most useful approaches to code, humanities scholars can help by conjecturing on the meaning of code to all those who encounter it both directly by reading it or indirectly by encountering the effects of the programs it creates. In effect, I am proposing that we can read and explicate code the way we might explicate a work of literature in a new field of inquiry that I call Critical Code Studies (CCS).

Codework critics and artists have operated on the cusp of this analysis. Cayley's emphasis, for example, is "the role of code in literal art" but not the role of code in all software objects, even those not intended to be literary. The focus of CCS is not literature made of code or code that is literature, although these may benefit from its techniques. Rather, I propose that code itself is a cultural text worthy of analysis and rich with possibilities for interpretation. Of course, some code will yield more opportunities for interpretation than others, but that can also be a product of the critics themselves.

Nonetheless, codework critics and artists offer avenues and entryways. "Code has its own structures, vocabularies and syntaxes," Cayley writes. These are the languages that codework critics can learn. Code "functions, typically, without being observed, perhaps even as representative of secret workings, interiority, hidden processes," he explains. This hidden language is what we can uncover, explore, access, and engage. "There are division and distinctions between what the code is and does," he notes. This relationship with the coding structures and their processes is what we must trace, interrogate, and map. Although Cayley is writing about codework, he argues against reading code merely as "a chain of floating signifiers," a term he borrows from N. Katherine Hayles. For Cayley, however, the "code" of most importance is that which makes the signifiers flicker, that which is "programming the signifier." However, that which causes the signifier to flicker is also a signifier.

Critical Code Studies (CCS) is an approach that applies critical hermeneutics to the interpretation of computer code, program architecture, and documentation within a socio-historical context. CCS holds that lines of code are not value-neutral and can be analyzed using the theoretical approaches applied to other semiotic systems in addition to particular interpretive methods developed particularly for the discussions of programs. Critical Code Studies follows the work of Critical Legal Studies, in that it practitioners apply critical theory to a functional document (legal document or computer program) to explicate meaning in excess of the document's functionality, critiquing more than merely aesthetics and efficiency. Meaning grows out of the functioning of the code but is not limited to the literal processes the code enacts. Through CCS, practitioners may critique the larger human and computer systems, from the level of the computer to the level of the society in which these code objects circulate and exert influence.

Rather than creating a language separate from the work of programmers, Critical Code Studies will build on pre-existing terminology and analysis used within the programming community. Much of the current examination of code seems to revolve around efficiency, reusability, and modularity. My own critical approach will stress meaning, implication, and connotation, though not in terms of a self-contained system of meaning but with respect to the broader social contexts. While a computer scientist might argue for or against various pragmatic approaches, scholars of CCS will analyze the extra-functional significance of the code.

Just as developing new media technologies requires the collaboration of artists and programmers, the area of CCS will require the artful combination of knowledge of programming languages and knowledge of interpretive approaches. These analytic projects will require programmers to help open up the contents and workings of programs, acting as theorists along with other scholars, as they reflect on the relationships between the code itself, the coding architecture, the functioning of the code, and specific programming choices or expressions, to that which it acts upon, outputs, processes, and represents.

Like literary analysis, CCS is an interpretive process rather than an instrumentally proscriptive or solely descriptive process. While other branches (lines of flight) of code studies may be concerned with pragmatics, CCS focuses on meaning, read from the often collaborative and certainly iterative performance that is coding. If as Wendy Hui Kyong Chun has announced, "software is ideology" we might also say computer code is ideology, yet an ideology that is doubly hidden by our illiteracy and by the very screens on which its output delights and distracts ("Stroking" 207). While open source embodies a particular ideology that makes code accessible for analysis, CCS will analyze more than open source programs. CCS will look broadly at choices in paradigms (such as object-oriented approaches) and closely to specific lines of code.

Critical Code Studies can draw upon the works of scholars who have begun to interpret code, particularly Matthew Fuller's enthralling, forthcoming collection entitled Software Studies. Fuller's text, discussed in more detail below, offers a lexicon for the interpretation of software and its role in society. If Critical Code Studies names a set of approaches, or an interpretive attitude, to code, Software Studies offers "a handbook of supplements to some of the key standard objects of computer science, programming and software culture" (Introduction) Indeed, Fuller's sense of software relates well to perspective of CCS, as he argues that "software can be seen as an object of study and an area of practice for kinds of thinking and areas of work that have not historically 'owned' software, or indeed often had much of use to say about it." Software Studies gestures towards a more formalized practice of (and quite a few tools for engaging in) Critical Code Studies.

I do not want to limit Critical Code Studies to the study of code written as literature (as in Perl Poetry), although this interpretation is certainly related. The focus of CCS is not on making code that has aesthetic value and additional meaning but a view of code as already having meaning beyond its functionality since it is a form of symbolic expression and interaction. Nonetheless, analyses of Perl Poetry, and codework in general, perform CCS and model methods for interpreting specific lines of code.

I also do not want to confuse CCS with literate programming, as outlined by Donald Knuth (1984). "Instead of imagining that our main task is to instruct a computer what to do, let us concentrate rather on explaining to human beings what we want a computer to do." Knuth continues, "The practitioner of literate programming can be regarded as an essayist, whose main concern is with exposition and excellence of style." Again, this aesthetic, meant to instruct programmers, will no doubt give critics something to interpret, but it is not the same as investigating the meaning of programs themselves.

Fundamental to CCS is the assumption that code is a social, semiotic system employing grammar and rhetoric. As Rita Raley argues, "Code may in a general sense be opaque and legible only to specialists, much like a cave painting's sign system, but it has been inscribed, programmed, written. It is conditioned and concretely historical" (24). A scholar at the Institute for Cultural Research in Lancaster, UK, Adrian MacKenzie writes, "Code is written and run within situated practices, with reference to particular domains, and within particular orderings and disorderings of collective life. Its forms and abstractions are attached to lives" (19). The way this sign system circulates within actor-networks of computers and machines is also the way it develops connotations worthy of interpretation.

In his entry in Software Studies, Friedrich Kittler warns against diluting, and thereby mystifying and universalizing, the term "code." Kittler warns, "Codes - by name and by matter - are what determine us today, and what we must articulate if only to avoid disappearing below them completely.The notion of code is as over-used as it is questionable." (forthcoming). In the entry, "Code (or, How You Can Write Something Differently)," Kittler traces the derivation of the word back to codex and further to codicilla, "the small tablets of

stripped wood coated with wax in which letters could be inscribed." In its later form, "codex," the word signifies "simply the name of the bound book of law." Code thus becomes the means and medium of long-distance control. Kittler follows the term from classical empires to nation states to the moment when it becomes synonymous with "cipher."

In this historical narrative, codes will prove to be central to subjugation as the military motto changes from "Command, Control, Communication, Intelligence" "C^4" "Command, control, communication, Computers." Kittler writes, "Today, technology puts the code into practice of realities, that is to say: encodes the world." Central to control is the ability to make "code" a ubiquitous, universal operator. Moreover, he adds,

But perhaps code means nothing more than codex did at one time: the law of precisely that empire which holds us in subjection and forbids us even to articulate this sentence. At all events, the major research institutions which stand to profit most from such announcements proclaim with triumphant certainty that there is nothing in the universe, from the virus to the Big Bang, which is not code.

Here, the military-industrial complex employs the codes for control, and by making "code" the essence of all life, the few elite, literate researchers establish themselves as the only mediators. To regain command and control of the term, Kittler sets out to restrain it within a technical definition. He announces, "Only alphabets in the literal sense of modern mathematics should be known as codes, namely one-to-one, finite sequences of symbols, kept as short as possible but gifted, thanks to a grammar, with the incredible ability to infinitely reproduce themselves." Critical Code Studies may observe some of Kittler's suggested restraint, keeping code from becoming a controlling, transcendental imaginary, but in examining code in context our approach will look towards and beyond characters to interpret gestures, performances, and possibilities.

What can be interpreted?

Everything. The code, the documentation, the comments, the structures - all will be open to interpretation. Greater understanding of (and access to) these elements will help critics build complex readings. In "A Box Darkly," discussed below, Nick Montfort and Michael Mateas counter Cayley's claim of the necessity for executability, by acknowledging that code can be written for programs that will never be executed. Within CCS, if code is part of the program or a paratext (understood broadly), it contributes to meaning. I would also include interpretations of markup languages and scripts, as extensions of code. Within the code, there will be the actual symbols but also, more broadly, procedures, structures, and gestures. There will be paradigmatic choices made in the construction of the program, methods chosen over others and connotations.

In addition to symbols and characters in the program files themselves, paratextual features will also be important for informed readers. The history of the program, the author, the programming language, the genre, the funding source for the research and development (be it military, industrial, entertainment, or other), all shape meaning, although any one reading might emphasize just a few of these aspects. The goal need not be code analysis for code's sake, but analyzing code to better understand programs and the networks of other programs and humans they interact with, organize, represent, manipulate, transform, and otherwise engage.

In his essay, Cayley recognizes multiple audiences for the code. At first glance, the only audience appears to be the machine itself, but as mentioned, there are humans as well. First, the programmer must read the code even while compositing it. Second, other programmers may read the code. Third, non-programmers, such as project managers or audiences of interdisciplinary conferences, may read the code. The code may also end up in the hands of hackers. Even the machine on which the computer runs may prove to be multiple audiences, as parts of the code are passed to other processes and other parts of the machine.

To return to our opening example, we might note that the Lisp "Hello World" did not appear out of nowhere. It was written by Mark Singletary, a programming architect, variously employed by NASA and other software firms. His apparent specialty, usability, or the development of easily read human-computer interfaces, the facilitation of interaction between human and machine realms. The relationship between code and coder, the program that makes computers speak to a "world" and the programmer who works (for the government-funded, space-exploration C^4 institution) to make computers speak more clearly, creates meaning. Moreover, Singletary submitted this "beginner" code to the Association for Computing Machinery (ACM) "Hello-World" Project, a project that in its very being suggests additional audiences for practice code. What does it mean for ACM to collect samples of "first exposure to a new language"? What does it mean when an exercise, a rite of passage, is anthologized?

Suddenly this practice code has a showcase, even earning the status of "Geek Site of the Day," February 1, 1996. Thus the "World" the program greets has expanded to include other programmers at computers with Internet access (qualifiers that constrain the world to an absurdly tiny portion of the Earth's population). Moreover, even the programming assignment "Hello World" is not without its context and discontents. Ralph Westfall of Cal Poly Pomona has argued that teaching "Hello World" as an exercise can harm the learning process, inciting a further debate. Certainly, this program is not just a piece of code processed by the computer. It is an artifact circulating in a broader exchange. All these contextualizing aspects potentially affect the way "Hello World" means.

Why study the paint?

When I have spoken to programmers and computer science professors about this idea, they ask a lot of questions about what a critical approach would mean. Our discussions suggest to me once again that the approaches of semiotic analysis common to critical theory in the humanities are foreign to the disciplinary emphases of programming. So, a movement toward consideration of the code as a semiotic unit that can be explicated, interpreted, read, and perhaps misread, seems a significant change.

Playing the devil's advocate, Jeremy Douglass offered a parallel with another art form. He asked, what do you say to the person who says, "Why do I have to study film-development processes, if they do not have a perceivable affect (to the naked, untrained eye) on the image that results. Would you also call for a school of film-development studies?" First of all, I do not doubt that there are people who analyze film development and do interpretations of photography at the chemical level. Second, and more relevant here, unlike film processing, code is not merely a means or procedure, it is a text, which is often accessible after the program has executed. This text can be read, understood, and of course, altered. The clearer analogy is the analysis of a musical score, a play script, blueprints, circuit diagrams, or any print text, since none of these can be processed or executed without being read. Or perhaps code is more like a spell or incantation: to read it is to cast it. In any event, when Raley invokes Adorno regarding the emphasis on the score over the music, she cautions us against emphasizing the code at the expense of the processes, the performance of the code. As Cayley notes, in these new media objects, code is not merely instrumental but rather a performative, transformative, mediating process.

The ABCs of PERL

Key to Critical Code Studies will be the development in practitioners of programming literacy. Just as a student of literature might learn a foreign language or Flash, a student of CCS will only be able to access code by learning and mastering the language. Raley has speculated about computer languages counting for language requirements in undergraduate and graduate programs in the humanities (2002). Alternatively, an interpreter could collaborate with someone who is literate in the language, building networks of literature, and coalitions of meaning building. We might consider such fluency "cyborg literacy." Perhaps, computer scientists will even one day write translations. No doubt in textbooks on programming, they do.

Shouldn't we be interpreting machine language? Perhaps. If someone does, I would call it Critical Code Studies. Such projects have much in common with critiques of numerals, such as Brian Rotman's Signifying Nothing, which suggests that much can be made of nothingness. Equations have lives of their own, as texts such as David Bodanis' E=MCÂ² demonstrate. Where the line between mathematics and CCS is, I do not wish to declare. I would suggest that there are qualities that give a program more footholds for interpretation.

Code (most of it) is not poetry

Loss PequeÃ±o Glazier's article, "Code as Language" argues that "if language is defined as written symbols organized into combinations and patterns to express and communicate thoughts and feelings - language that executes - then coding is language" (1). Glazier is establishing code as a creative, even literary act, a kind of inscription or "thinking through thought" (1). In his essay, Glazier offers additional objects for CCS analysis, including the amount of processor speed a particular operation requires. Regarded as more than mere utilitarian commands utilitarian commands, code presents signs of "humor, innovation, irony, double meanings, and a concentration on the play of language. It is the making of marks with a sense of marksmanship" (7). Again, key to CCS is an understanding that the act of encoding is not a question of finding one correct command out of a codex of commands, but of choosing (and at times creating) a particular combination of lines to build a structure that resonates and operates aesthetically, functionally, and even conceptually with the other discourse of encoded objects as well as mathematical and natural language discourse.

If code is language, does that make programs poetry? In "The Aesthetics of Generative Code," Geoff Cox, Alex McClean, and Adrian Ward compare code to poetry, as they develop some techniques for interpreting it. In their words, "Evidently, code works like poetry in that it plays with structures of language itself, as well as our corresponding perceptions" (Cox, McLean, and Ward). Like Cayley and others, they argue that code's meaning is bound up with its execution. However, their reading is not limited to the performance of the code, as they also stress "more purposeful arrangements of code by the programmer," noting for example that even the visual of code is for human readers, since "the same code could be expressed in any shape or arrangement and would run the same output." Nonetheless, in an exemplary CCS move, they examine the play of language from a Perl Poem in which the author uses "=" ("a string comparison operator") instead of "eq" ("a numeric one"). To further argue that code is a linguistic form for human audiences, they present their paper as a "conditional statement" of code (if, then), wherein an "if" is followed by their abstract as the {condition} and the essay body is marked as the {statement}. While they make a strong case for a human poetics in the construction of code, to suggest that all code adheres to an aesthetic or works towards readability, however, is to overstate the case.

Excluding the few art-objects written as poetry, most code is worlds away from literature, film, and visual art and more similar to legal code, inasmuch as the text is functional. Unlike legal code, however, computer code is typically presumed to be unseen by human eyes. Nonetheless, except in cases in which the computer generates code that its programmers and users never see, computer code is often seen by other programmers who must rework it. Further, even if the code is only seen by its programmer, like a diary locked away or letters never sent, it is still a semiotic expression. (Of course, computer-generated code presents the author as cyborg, with the human author at least one step removed from the produced code. Surely, humans will need to enlist machines as collaborators for the interpretation of

some such code.) More importantly, and I hope this is not too vague, there are implications in the way a code tries to perform a function that bear the imprint of epistemologies, cultural assumptions about gender, race and sexuality; economic philosophies; and political paradigms. This brief litany does not begin to get at the more computer-specific issues.

Code frequently has multiple authors, mostly uncited. To use a common algorithm could be thought of as using a screw. Mechanics do not cite the inventor of the screw every time they use one, although programmers at times attribute code to particular sources. Nonetheless, literary analysis has found other means of involving authorship, including the Foucauldian notions that the authors themselves are assemblages of influences.

Not all code is open to the public, despite the open source movements. It is unlikely that someone will soon be able to comment on the code of Microsoft Word, hegemonic though that program might be. However, researchers frequently publish their code with commentary and demonstrations, as in the case of Terry Winograd's SHRDLU.

Current Practitioners of Critical Code Studies

Critical Code Studies names a set of practices that are already under way, while offering a framework for developing the few examples already begun. Notable readings of code have been conducted by Raley, Montfort, Mateas, Noah Wardrip-Fruin, and contributors to the Software Studies collection (forthcoming). Most of the work until now has dealt with broad concepts regarding code. Too few critics have dealt with specific lines of code or aspects of programming languages even when analyzing codework. The critics reviewed below have begun the process of analyzing specific programming practices, computer languages, and the very symbols of code in the kinds of close readings necessary for CCS to be engaged in a thorough reading of these texts. Their essays help to frame CCS even while they demonstrate how something as small as a parenthesis can signify beyond its functional application.

Critics have already set the stage for this analytical project. Notably, Cayley and Glazier have declared that "programming is writing" (Hayles 2004: 80) and more recently that "encoding is writing" (Glazier 2006: 4). Cramer has written, "I believe it is a common mistake to claim that machine language would be only readable to machines and hence irrelevant for human art and literature and, vice versa, literature and art would be unrelated to formal languages" (2001). Although these scholars primarily focus on the relationship of code to literary objects, their statements certainly anticipate, incite, and inspire CCS.

Further, Mateas and Montfort make an intervention in discussions of code aesthetics, by analyzing "weird languages" and obfuscated code that cannot be explained by elegance or functionality. Obfuscated code is a program whose functioning has been rendered "more or less impenetrable, even when there is no commercial or practical reason for doing so" (1). Developed "as jokes or parodies," "Weird languages" or "esoteric programming languages"

are "designed to make legibility of any program difficult" (1, emphasis theirs). In their view, the existence of both of these objects "throws a wrench into the simplified theory of coding that would claim that coders must always strive for clarity" (1). Most notably, they introduce "double coding," a practice which produces source code that can be compiled or read in multiple contexts, for example, a program that can be compiled in multiple languages. Double coding becomes a means for describing the multiple meanings inherent in code-just as it is inherent in natural language-adding the computational realm as another milieu of signification.

Ultimately, they conclude that "All coding inevitably involves double coding. 'Good' code simultaneously specifies a mechanical process and talks about this mechanical process to a human reader" (10). Weird languages or obfuscated code are speaking to a human audience by parodying or commenting on coding practices. Like some codework, these objects are commentaries in executable form. For Montfort and Mateas, these two coding categories "suggest that coding can resist clarity and elegance to strive instead for complexity, can make the familiar unfamiliar, and can wrestle with the language in which it is written, just as much as contemporary literature" (10). As they make their arguments, they perform several close readings of obfuscated programs and programs written in weird languages, including versions of "Hello World."

"A Box Darkly" opens up the black box of software to interpretation. Through this intervention, Mateas and Montfort demonstrate that programmers can signify through stylistics and play in ways that go far beyond mere questions of efficiency and legibility, opening the analysis of code to other criteria than utility and grace of technique. Their interpretations are instructive in that they are deeply rooted in particular programming languages and practices. For example, they discuss a particular moment in obfuscation when "---" is used to represent 0 (the value of the numeric sign "-" subtracted from itself) (3). These examples illustrate the ways in which the code has been designed to "fool a reader," demonstrating both the intentionality behind coding poetics and the awareness of the human readers (4). However, while Mateas and Montfort focus on the moment "when a program is double-coded to have some literary meaning," I would argue that this is only one kind of coding object to be interpreted (10). Self-reflexive code objects, like codeworks offer avenues of critique of code at large just as self-reflexive literary objects offer meta-commentary on writing at large. Nonetheless, a coding object does not have to be self-reflexive, or even a purposeful commentary on coding, to be open for interpretation. Returning to their notion that all programs are "double coded," we find that all programs present the possibility and opportunity for (de)constructive interpretation. Even so, these objects that purposely play with the conventions of code are particularly rich subjects of analysis.

In her essay "Code.Surface || Code.depth," Rita Raley critiques a broader conceptual mythology that pervades discussions of code objects and interprets coding structures as means of illustration. Raley interrogates the notion of a "deep structure of the code," examining the metaphors and mythologies of "code," asking, "Is code then semiotic?" (5). Throughout the essay, Raley contrasts "the discourse of the foundational architecture of code that mines the depths to produce a geology (or mythology) of surface" with "the discourse of computational code in terms of inaccessible, inscrutable processes" (2). Raley identifies "the difference of the sign system of code" as "its executability, the operative transformation of a message from one symbolic form to another" (12). Yet it is this dual state as signs and operations that offers much to the play of interpretation of code that she performs. Here, Raley cites MacKenzie's "account of code in terms of linguistic performative indicating the gap between expression and execution" (12) Much of this preliminary work on code has tried to determine what constitutes this unique semiotic system, work that prepares the discussion of code analysis, but CCS needs techniques for developing particular coding structures.

The readings in Raley's essay offer several examples of interpreted signs from code. For her objects of study, Raley examines Ted Warnell's "Lascuax Symbol.ic," a poem written in HTML, or to be more exact, commented-out JavaScript within HTML tags (<!-- and -->), and various works by Cayley, along with other non-computational works. In addition to analyzing the way codeworks function when executed, Raley interprets specific coding symbols, the "double pipe, the logical 'or' condition (||)," which she has already set into play in her title. She also addresses the ironic use of JavaScript comment tags (//) in "Lascaux.Symbol.ic," symbols traditionally used to indicate the documentation for the program, the characters that will not be processed as code. (These symbols ironically proceed the line, "End the hiding here.") As in other examples in this essay, Raley's analyses of codeworks (along with the codeworks themselves) will provide useful models for CCS. The challenge will be to apply such readings to less intentionally artistic programs.

Fuller's Software Studies offers a major step towards such analysis. The entries provide interpretive frameworks, and several model specific Critical Code Studies techniques. Geoff Cox and Adrian Ward's entry on "Perl" examines the "porting" of William Blake's Poem "London" into that language. Other entries that perform close readings of code include Florian 's entry "language," Alison Adam's "Lists," Monfort's "Code," and Mateas' "Weird languages," the last two reprising and developing the arguments from the DAC paper mentioned above. Author of Gender, Ethics, and Information Technology, Adam discusses the power of Lisp, which could inflect the reading of "Hello World" that began this paper. As she argues, the longevity of Lisp language "captures something about the human condition and our need to make and manipulate lists to make sense of the world." Here she offers a reflection on the historical nature of lists and a subsequent reading of lists in Lisp, as echoes

19th century lists and gestures towards something "beyond." The treatment of the cultural history and meaning of these languages will help contextualize analyses of specific programs.

While several entries analyze specific lines of code, the collection of contemporary computer keywords offers other philosophical, historical, and literary reflections on notions that are crucial to Critical Code Studies. Fuller himself considers the notion of "Elegance." Lev Manovich meditates on "Import/Export" and Derek Robinson comments on "Variable." Manovich, had proposed a transition from "new media studies" to "software studies" in 2001 in The Language of New Media (48). Joasia Kryssa and Grzesiek Sedek also trace the conceptual origins of programming to the kitchen and recipes. The readings contextualize the conceptual tropes of programming and software within a broad historic, literary, and cultural analyses. The entries are guides and foundations of CCS.

Along with the other new media critics who have begun analyses of code, N. Katherine Hayles offers an engagement with the object-oriented language C++ in My Mother Was a Computer: Digital Subjects and Literary Texts. Although not examining one specific piece of code, Hayles develops a reading of object-oriented languages in relation to procedural languages motivated by similar concerns to Kittler's. For Hayles, "Code is not the enemy anymore than it is the savior. Rather code is increasingly positioned as language's pervasive partner. Implicit in the juxtaposition is the intermediation of human though and machine intelligence, with all the dangers, possibilities, liberations, and complexities this implies" (61). As argued by print-literacy advocates, liberation may be achieved through literacy in code. By pursuing a knowledge of these pervasive and powerful texts, "Strategies can emerge from a deep understanding of code that can be used to resist and subvert hegemonic control by megacorporations" (61). Within the operations of a critical literacy, entering the symbolic order of code, however, does not make one complicit in the systems that employ that code. As she explains,

Anxieties can arise when the operations of the computer are mystified to the extent that users lose sight of (or never know) how the software actually works, thus putting themselves at the mercy of predatory companies like Microsoft, which makes it easy (or inevitable) for users to accept at face value the metaphors the corporations spoon feeds them. (60)

This call from Hayles and others envisions a leserevolution of code, a moment when code comes to the people.

As many of these arguments have indicated, CCS will move beyond merely reading another set of symbols. Alan Sondheim, who coined the term "codework," has also opened doors to more sophisticated CCS. Notably in "On Code and Codework" (2006) Sondheim interrogates the relationship between "coding" and "encoding," marking one as the practice of the programmer and the other as the process of the program when handling input. Rather than

treating code as static method of signification only available to programmers, he talks about how the computer encodes input. His article gestures towards the possibilities for interrogating the epistemology of code moving beyond a simple application of semiotics.

Other versions of Critical Code Studies, though not all called such, have been appearing in dissertations, including the works of Noah Wardrip-Fruin, Jeremy Douglass, and myself. Wardrip-Fruin characterizes this operation as reading for process. Douglass applies the process directly to the programming languages of interactive fiction. Bradley Dilger and Jeff Rice have been assembling a collection with the working title, From A to <A>: Keywords in HTML and Writing, whose call requested papers "considering the complex relationship between writing and the markup and scripting languages" used for web documents. Although mark-up languages are not the same as other programming languages, this kind of study certainly fits my description of CCS. Graduate students are continuing to enter this area of research. Maurice J. Black has written one of the first dissertations on the issue. Also, notably, Paul Swartz, a Ph.D. candidate at Hampshire College, is currently pursuing the question: "what would it mean to treat computer software as a 'literature?'" Not doubt this is only the beginning of this interdisciplinary work.

Lastly, Critical Code Studies shares affinities with other types of what Hayles calls media-specific analysis. One brilliant Kittlerian example of this technique appears in Matthew Kirschenbaum's "Extreme Inscription - Towards a Grammatology of the Hard Drive" (2004) in which he reads significance from not only the structure of computer hardware memory but the social role of "storage" itself. Such readings perform read hardware along the sames lines I call for in software, which are by no means separate or discrete elements of computation. Cramer disrupts this binary, arguing, "If the duality of software and hardware needs to be suspended, it follows that the notion of software as immaterial versus hardware as material must be suspended, too" (2005: 124). At the limits of code studies are, indeed, hardware studies. As Chun points out, "Software cannot be physically separated from hardware, only ideologically" (19). At risk of ignoring "the significance of hardware and extramedial representation," CCS must at times engage with circuits and microprocessors (18). Nonetheless, just as some essays deal with texts and others with books, not every analysis of code requires full engagement with that which conducts and embodies it as electrical signals.

// Cautionary Comments

One of my early experiences with CCS may serve as a cautionary tale. When I proposed an earlier version of CCS to a group of programmers, one of them asked if I could prove CCS by applying these methodologies to a program with low natural language content, even in terms of input and output. The reviewer suggested Quicksort, an algorithm used for quickly ordering groups of numbers through a "divide and conquer" approach. In my early attempts at what Wardrip-Fruin would equate to interpreting a stop sign, I suggested Quicksort as a

metaphor for social organization in communities, drawing out an analogy for the way a neighborhood street or even highway may serve to divide and conquer a demographic. However, though my analysis said something about neighborhood hierarchies, it offered little insight on Quicksort itself, nor did it draw from Quicksort a lesson about the society from which it came. My main error was analyzing Quicksort aside from its historical, material, social context. For an algorithm such as Quicksort, the code meets the social realm at the site of its implementation and in the context of the application in which it is used. I was not engaging the cultural origins of Quicksort within the history of computation or even a particular incarnation of Quicksort in a language (I was using pseudocode). Without discussing the human context of the Quicksort code in terms of authorship, use, development, circulation, or operations, I was left with little more than a process

Interpretation requires reading an object in its (post)human context through a particular critical lens. This context involves human machines operating in actor-networks. Thus, a simple looping subroutine, say, might remind one of the eternal return of the repressed, but unless that metaphor has significance with respect to the particular, material context of the script itself, the interpretation is not very significant. However, if one found a recursive loop in a program designed to psychoanalyze its users, perhaps a connection could be drawn between recursion and the psychoanalytic view of the return of the repressed. Thus, while these computer programs are quite meaningful, they will only yield their meaning to the extent to which we interrogate their material and social-historical context to elucidate some aspect of our cultures.

Throughout this proposal, I have tried to highlight the moments in other critical writings when they have read specific coding elements because this is the kind of interpretation that is most lacking in analyses of codework and code. Cultural critics often speak abstractly of coding practices or the processes of code without getting to the code itself. My emphasis, however, should not understate the need for interpreting these structures with an eye towards fundamental human concerns, concerning race, ethnicity, gender, and sexuality; concerning the military-industrial-entertainment and academic complex; concerning surveillance and control over electronic systems-to name but a few. Such readings will prove critical in the analysis of systems that do everything from facial recognition to tactical to simulation of political systems to modeling human reasoning about peoples and places.

The Moment is Critical

Many developments have transpired to make this the moment for Critical Code Studies. In the first place, higher-level programming languages are becoming readable to a wider population. The interactive fiction language Inform 7 is just one example. Secondly, a growing number of critical theorists (and artists) are becoming literate in programming languages. Third, the programming community is beginning to consider aesthetics in a new way, which will no doubt lead to new styles of programming artistry. Fourth, colleges and

universities are continuing to develop more programs in the humanistic analysis of science and scientific culture. Not finally, works of code in art, art in code, codeworks, and electronic literature are proliferating, offering more of the moments of connection between represented signs and programmatic signs, as in the objects Cayley described earlier in ebr. Code increasingly shapes, transforms, and limits our lives, our relationships, our art, our cultures, and our civic institutions. It is time to take code out from behind quotation marks, to move beyond execution to comment, to document, and to interpret. Let us make the code the text.

Acknowledgments:

I am deeply indebted to several people who guided my understanding of CCS and its limitations. Stephanie August and Philip Dorin of the Department of Electrical Engineering and Computer Science at Loyola Marymount University helped me to articulate the premise using the terms of computer science debate. I am especially grateful to Stephanie for presenting the paper at the UC Irvine conference. Todd Millstein of Computer Science at UCLA helped me to anticipate boundaries of the study. Rita Raley offered me the pre-publication version of her text and also organized the MLA panel (2006) on reading code. Toby Miller led me to the leserevolution, and N. Katherine Hayles directed me to Alan Sondheim and other Critical Code scholars. Matthew Fuller generously shared Software Studies prior to its debut. Jeremy Douglass, my collaborator on Writer Response Theory, offered many questions, suggestions, and objections to help develop the idea. Arthur and James Marino provided invaluable proofreading. Jan Van Looy encoded the essay beautifully. Thanks, finally, go to Joseph Tabbi for his editorial guidance.

Portions of this paper originally appeared in the Critical Code Studies post on Writer Response Theory cited below.

Works Cited

Aarseth, Espen. Cybertext. Johns Hopkins UP, Baltimore, 1997.

Adam, Alison. "Lists" Software Studies.

Black, Maurice J. The Art of Code. Ph.D. Dissertation. University of Pennsylvania, 2002.

Bodanis, David. E=MC2: A Biography of the World's Most Famous Equation. New York: Berkley Trade, 2001.

Carlyle, Leslie. "Van Gogh's Paint-Was it Special?" Conservation Matters Lecture. Getty Museum. Los Angeles, California. 19 October, 2006.

Cayley, John. "The Code is not the Text (unless it is the text)" Electronic Book Review. 10 September 2002. 1 April 2005.

Chun, Wendy Hui Kyong. Control and Freedom: Power and Paranoia in the Age of Fiber Optics. Cambridge: MIT Press, 2006.

-- "Stroking the Keys" Sexuality in the Age of Fiber Optics.

Cox, Geoff, Alex McLean, and Adrian Ward. "The Aesthetics of Generative Code" Generative Art 00 conference. Politecnico di Milano, Italy, 2001. 24 November 2006.

Cox, Geoff and Adrian Ward. "Perl" Software Studies.

Cramer, Florian. "Digital Code and Literary Text" Beehive. 4:3. 2001. 4 June 2005.

-- Words Made flesh: Code, Culture, Imagination 6/2005, Media Design Research, Piet Zwart Institute Rotterdam, Netherlands, 2005. 23 November 2006.

Douglass, Jeremy. Personal Interview. 10 September 2006.

Fitzpatrick, Peter and Alan Hunt. "Critical Legal Studies: Introduction"Journal of Law and Society. (14:1) 1987. 1-3.

Fuller, Matthew, ed. Software Studies. Cambridge: MIT Press, forthcoming 2007.

-- "Elegance" Software Studies.

Glazier, Loss PequeÃ±o. "Code As Language" Leonardo Electronic Almanac. 14:05-06. September 2006. 2 November 2006.

-- Digital Poetics: The Making of E-Poetries. Tuscaloosa: University of Alabama Press, 2001.

Graham, Peter S. "Building the Digital Research Library: Preservation and Access at the Heart of Scholarship" Folett Lecture Series. UKOLN. Leicester University, 19 March 1997. Accessed 10 March 2006.

Hayles, N. Katherine. My Mother Was a Computer: Digital Subjects and Literary Texts. Chicago: The U of Chicago P, 2005.

-- "Print is Flat, Code is Deep: The Importance of Media Specific Analysis" Poetics Today. (25:1) 2004. 67-90.

Hofstadter, Douglas R. and the Fluid Analogies Research Group. Fluid Concepts and Creative Analogies: Computer Models of the Fundamental Mechanisms of Thought. New York: Basic Books, 1995.

Kirschenbaum, Matthew G. "Extreme Inscription -- Towards a Grammatology of the Hard Drive" TEXT Technology. 91:2. 2004. 16 October 2006.

Kittler, Friedrich. "Code (or, How You Can Write Something Differently)" Trans. Tom Morrison with Florian Cramer. Software Studies.

Knuth, Donald. "Literate Programming (1984)" Center for the Study of Language and Information, Stanford, California, 1992.

Kress, Gunther and Robert Hodge. Language as Ideology. Boston: Routledge & Kegan Paul, 1979.

Kryssa, Joasia and Grzesiek Sedek. "Source Code" Software Studies.

Latour, Bruno. Reassembling the Social: An Introduction to Actor-Network Theory. Oxford: Oxford UP: 2005.

MacKenzie, Adrian. "The Problem of Computer Code: Leviathan or Common Power?" Institute for Cultural Research, Lancaster University. 10 August 2006.

McCarthy, John. "The History of Lisp" 12 February 1979. 20 May 2005.

Manovich, Lev. "Import/Export" Software Studies.

-- The Language of New Media. Cambridge, Mass.: MIT P, 2001.

Marino, Mark. "Critical Code Studies" ISR Graduate Student Research Forum. UC Irvine. June 3, 2005.

-- "Critical Code Studies 2" 4 Apr. 2005. WRT: Writer Response Theory. 4 April 2005.

Mateas, Michael and Nick Montfort. "A Box, Darkly: Obfuscation, Weird Languages, and Code Aesthetics" Digital Arts and Culture. Copenhagen, 2005.

Mateas, Michael. "Weird Languages" Software Studies.

Miller, Toby. The Well-Tempered Self: Citizenship, Culture, and the Postmodern Subject. Parallax: Re-Visions of Culture and Society. Ed. Gerald Prince Stephen G. Nichols, and Wendy Steiner. Baltimore: The Johns Hopkins UP, 1993.

Montfort, Nick. "Obfuscated Code" Software Studies.

Raley, Rita. "Code.surface || Code.depth" forthcoming Dichtung Digital February 2007.

-- "Interferences: [Net.Writing] and the Practice of Codework" Electronic Book Review. 8 September 2002. 1 April 2005.

Robinson, Derek. "Variable" Software Studies.

Rotman, Brian. Signifying Nothing: The Semiotics of Zero. Stanford UP: Stanford, 1987.

Searle, John. "Minds, Brains, and Programs" Behavioral and Brain Sciences 3. 417-424

Seiwald, Christopher. "Seven Pillars of Pretty Code" Perforce. 17 March 2005.

Shklovsky, Victor. "Art as Technique." Russian Formalist Criticism: Four Essays. Trans. Lee T. Lemon and Marion J. Reis. Lincoln: University of Nebraska Press, 1965, 3-24.

Sierra, Kathy. "Code Like a Girl" Creating Passionate Users. 29 March 2006. 15 October 2006.

Singletary, Mark. "Hello World" ACM 'Hello World' Project. Lousiana Tech ACM. Ed. Himanshu Chhetri, Leopolf Keiser, Chuka Okoye, Estabon Fiallos. last updated: 26 October 2005. 4 March 2005.

Sondheim, Alan. "Part 1: On Code and Codework" Plantext Tools. 2005. 6 March 2006.

Swartz, Paul. "Division III Proposal: Programs as Literature" 16 September 2006. 1 November 1 2006.

Thomas, Douglas. Personal Interview. 12 November 2005.

Turkle, Sherry. Life on the Screen: Identity in the Age of the Internet. New York: Touchstone, 1995.

Tushnet, Mark. "Critical Legal Studies: a Political History" The Yale Law Journal. (110:5) 1991. 1515-1544. 3 June 2005.

Westfall, Ralph. "Technical Opinion: Hello, World considered harmful" Communications of the ACM. (44:10) 2001. 129-130

Cite this Essay:

. “Critical Code Studies”, Electronic Book Review, December 4, 2006, .
Readers wishing to respond to an essay in *ebr* may send ripostes or short glosses to the journal’s Managing Editor, Will Luers.